EE 467 Lab 6: Ensemble Learning and Random Forest

Welcome to lab 6 of EE 467! Today we are going to learn and try out **ensemble learning** algorithms, which make use of multiple learning algorithms to achieve better performance than any of them. We will apply four kinds of common ensembles to the Kaggle credit card fraud detection problem: **voting, bagging, boosting and stacking**. We will also try **random forest** learning, which is a special kind of bagging ensemble that consists of decision trees. Like the previous lab, all algorithms are evaluated by **accuracy, precision, recall and F1-score**.

Pre-processing / Feature Extraction

Let's start from the end of lab 5. First of all, we will load the credit card transaction dataset and re-do all the feature scaling and dataset splitting steps in the last lab:

In [1]:
```python
import tarfile

with tarfile.open("credit-card.tar.xz", "r:xz") as tar:
    tar.extractall()
```

In [2]:
```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, RobustScaler

## [ Loading Dataset ]
print("Loading Kaggle credit card transactions dataset...")
df = pd.read_csv("./creditcard.csv")

## [ Feature Scaling ]
print("Scaling transaction time and amount features...")
# Convert transaction time from seconds to hours in a day
df["Time"] = (df["Time"]/(60*60))%24
# Scale time features with StandardScaler (suitable for normally distributed data)
df["Time"] = StandardScaler().fit_transform(df["Time"].values[:, None])
# Scale amount with RobustScaler (robust to outliers, useful for skewed distributions
df["Amount"] = RobustScaler().fit_transform(df["Amount"].values[:, None])

## [ Feature-label / train-test splits ]
print("Performing feature-label / train-test splits...")

# Get feature and label values from original dataset
feat_all = df.drop(["Class"], axis=1).values
y_all = df["Class"].values

# Split samples into training and test sets
feat_train, feat_test, y_train, y_test = train_test_split(
    feat_all, y_all, test_size=0.4, random_state=0
)

print("Completed.")
```

```
A module that was compiled using NumPy 1.x cannot be run in
NumPy 2.4.1 as it may crash. To support both 1.x and 2.x
versions of NumPy, modules must be compiled with NumPy 2.0.
Some module may need to rebuild instead e.g. with 'pybind11>=2.12'.

If you are a user of the module, the easiest solution will be to
downgrade to 'numpy<2' or try to upgrade the affected module.
We expect that some modules will need time to support NumPy 2.

Traceback (most recent call last):  File "<frozen runpy>", line 198, in _run_module_a
s_main
  File "<frozen runpy>", line 88, in _run_code
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel_launcher.py", line 17, in
<module>
    app.launch_new_instance()
  File "c:\Users\kelly\anaconda\Lib\site-packages\traitlets\config\application.py", l
ine 992, in launch_instance
    app.start()
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelapp.py", line 736,
in start
    self.io_loop.start()
  File "c:\Users\kelly\anaconda\Lib\site-packages\tornado\platform\asyncio.py", line
195, in start
    self.asyncio_loop.run_forever()
  File "c:\Users\kelly\anaconda\Lib\asyncio\base_events.py", line 607, in run_forever
    self._run_once()
  File "c:\Users\kelly\anaconda\Lib\asyncio\base_events.py", line 1922, in _run_once
    handle._run()
  File "c:\Users\kelly\anaconda\Lib\asyncio\events.py", line 80, in _run
    self._context.run(self._callback, *self._args)
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 516,
in dispatch_queue
    await self.process_one()
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 505,
in process_one
    await dispatch(*args)
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 412,
in dispatch_shell
    await result
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 740,
in execute_request
    reply_content = await reply_content
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\ipkernel.py", line 422, i
n do_execute
    res = shell.run_cell(
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\zmqshell.py", line 546, i
n run_cell
    return super().run_cell(*args, **kwargs)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3024, in run_cell
    result = self._run_cell(
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3079, in _run_cell
    result = runner(coro)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\async_helpers.py", lin
e 129, in _pseudo_sync_runner
    coro.send(None)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3284, in run_cell_async
    has_raised = await self.run_ast_nodes(code_ast.body, cell_name,
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3466, in run_ast_nodes
    if await self.run_code(code, result, async_=asy):
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3526, in run_code
    exec(code_obj, self.user_global_ns, self.user_ns)
  File "C:\Users\kelly\AppData\Local\Temp\ipykernel_119260\4105771299.py", line 1, in
<module>
    import pandas as pd
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\__init__.py", line 39, in <m
odule>
    from pandas.compat import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\compat\__init__.py", line 2
7, in <module>
    from pandas.compat.pyarrow import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\compat\pyarrow.py", line 8,
in <module>
    import pyarrow as pa
  File "c:\Users\kelly\anaconda\Lib\site-packages\pyarrow\__init__.py", line 65, in <
module>
    import pyarrow.lib as _lib
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: _ARRAY_API not found
A module that was compiled using NumPy 1.x cannot be run in
NumPy 2.4.1 as it may crash. To support both 1.x and 2.x
versions of NumPy, modules must be compiled with NumPy 2.0.
Some module may need to rebuild instead e.g. with 'pybind11>=2.12'.

If you are a user of the module, the easiest solution will be to
downgrade to 'numpy<2' or try to upgrade the affected module.
We expect that some modules will need time to support NumPy 2.

Traceback (most recent call last):  File "<frozen runpy>", line 198, in _run_module_a
s_main
  File "<frozen runpy>", line 88, in _run_code
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel_launcher.py", line 17, in
<module>
    app.launch_new_instance()
  File "c:\Users\kelly\anaconda\Lib\site-packages\traitlets\config\application.py", l
ine 992, in launch_instance
    app.start()
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelapp.py", line 736,
in start
    self.io_loop.start()
  File "c:\Users\kelly\anaconda\Lib\site-packages\tornado\platform\asyncio.py", line
195, in start
    self.asyncio_loop.run_forever()
  File "c:\Users\kelly\anaconda\Lib\asyncio\base_events.py", line 607, in run_forever
    self._run_once()
  File "c:\Users\kelly\anaconda\Lib\asyncio\base_events.py", line 1922, in _run_once
    handle._run()
  File "c:\Users\kelly\anaconda\Lib\asyncio\events.py", line 80, in _run
    self._context.run(self._callback, *self._args)
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 516,
in dispatch_queue
    await self.process_one()
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 505,
in process_one
    await dispatch(*args)
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 412,
in dispatch_shell
    await result
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 740,
in execute_request
    reply_content = await reply_content
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\ipkernel.py", line 422, i
n do_execute
    res = shell.run_cell(
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\zmqshell.py", line 546, i
n run_cell
    return super().run_cell(*args, **kwargs)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3024, in run_cell
    result = self._run_cell(
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3079, in _run_cell
    result = runner(coro)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\async_helpers.py", lin
e 129, in _pseudo_sync_runner
    coro.send(None)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3284, in run_cell_async
    has_raised = await self.run_ast_nodes(code_ast.body, cell_name,
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3466, in run_ast_nodes
    if await self.run_code(code, result, async_=asy):
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3526, in run_code
    exec(code_obj, self.user_global_ns, self.user_ns)
  File "C:\Users\kelly\AppData\Local\Temp\ipykernel_119260\4105771299.py", line 1, in
<module>
    import pandas as pd
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\__init__.py", line 62, in <m
odule>
    from pandas.core.api import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\api.py", line 9, in <mo
dule>
    from pandas.core.dtypes.dtypes import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\dtypes\dtypes.py", line
24, in <module>
    from pandas._libs import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pyarrow\__init__.py", line 65, in <
module>
    import pyarrow.lib as _lib
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: _ARRAY_API not found
A module that was compiled using NumPy 1.x cannot be run in
NumPy 2.4.1 as it may crash. To support both 1.x and 2.x
versions of NumPy, modules must be compiled with NumPy 2.0.
Some module may need to rebuild instead e.g. with 'pybind11>=2.12'.

If you are a user of the module, the easiest solution will be to
downgrade to 'numpy<2' or try to upgrade the affected module.
We expect that some modules will need time to support NumPy 2.

Traceback (most recent call last):  File "<frozen runpy>", line 198, in _run_module_a
s_main
  File "<frozen runpy>", line 88, in _run_code
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel_launcher.py", line 17, in
<module>
    app.launch_new_instance()
  File "c:\Users\kelly\anaconda\Lib\site-packages\traitlets\config\application.py", l
ine 992, in launch_instance
    app.start()
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelapp.py", line 736,
in start
    self.io_loop.start()
  File "c:\Users\kelly\anaconda\Lib\site-packages\tornado\platform\asyncio.py", line
195, in start
    self.asyncio_loop.run_forever()
  File "c:\Users\kelly\anaconda\Lib\asyncio\base_events.py", line 607, in run_forever
    self._run_once()
  File "c:\Users\kelly\anaconda\Lib\asyncio\base_events.py", line 1922, in _run_once
    handle._run()
  File "c:\Users\kelly\anaconda\Lib\asyncio\events.py", line 80, in _run
    self._context.run(self._callback, *self._args)
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 516,
in dispatch_queue
    await self.process_one()
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 505,
in process_one
    await dispatch(*args)
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 412,
in dispatch_shell
    await result
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\kernelbase.py", line 740,
in execute_request
    reply_content = await reply_content
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\ipkernel.py", line 422, i
n do_execute
    res = shell.run_cell(
  File "c:\Users\kelly\anaconda\Lib\site-packages\ipykernel\zmqshell.py", line 546, i
n run_cell
    return super().run_cell(*args, **kwargs)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3024, in run_cell
    result = self._run_cell(
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3079, in _run_cell
    result = runner(coro)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\async_helpers.py", lin
e 129, in _pseudo_sync_runner
    coro.send(None)
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3284, in run_cell_async
    has_raised = await self.run_ast_nodes(code_ast.body, cell_name,
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3466, in run_ast_nodes
    if await self.run_code(code, result, async_=asy):
  File "c:\Users\kelly\anaconda\Lib\site-packages\IPython\core\interactiveshell.py",
line 3526, in run_code
    exec(code_obj, self.user_global_ns, self.user_ns)
  File "C:\Users\kelly\AppData\Local\Temp\ipykernel_119260\4105771299.py", line 1, in
<module>
    import pandas as pd
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\__init__.py", line 62, in <m
odule>
    from pandas.core.api import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\api.py", line 28, in <m
odule>
    from pandas.core.arrays import Categorical
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\arrays\__init__.py", li
ne 1, in <module>
    from pandas.core.arrays.arrow import ArrowExtensionArray
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\arrays\arrow\__init__.p
y", line 5, in <module>
    from pandas.core.arrays.arrow.array import ArrowExtensionArray
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\arrays\arrow\array.py",
line 50, in <module>
    from pandas.core import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\ops\__init__.py", line
8, in <module>
    from pandas.core.ops.array_ops import (
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\ops\array_ops.py", line
56, in <module>
    from pandas.core.computation import expressions
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\computation\expression
s.py", line 21, in <module>
    from pandas.core.computation.check import NUMEXPR_INSTALLED
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\core\computation\check.py",
line 5, in <module>
    ne = import_optional_dependency("numexpr", errors="warn")
  File "c:\Users\kelly\anaconda\Lib\site-packages\pandas\compat\_optional.py", line 1
35, in import_optional_dependency
    module = importlib.import_module(name)
  File "c:\Users\kelly\anaconda\Lib\importlib\__init__.py", line 126, in import_modul
e
    return _bootstrap._gcd_import(name[level:], package, level)
  File "c:\Users\kelly\anaconda\Lib\site-packages\numexpr\__init__.py", line 24, in <
module>
    from numexpr.interpreter import MAX_THREADS, use_vml, __BLOCK_SIZE1__
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: _ARRAY_API not found
```

```
Loading Kaggle credit card transactions dataset...
Scaling transaction time and amount features...
Performing feature-label / train-test splits...
Completed.
```

During this lab we will use two utility functions from `lab_7_util`. The `timeit` function, which you should be fairly familiar with, times Python operations happening within the corresponding `with` block. The `evaluate_model` function evaluates a trained classification model on the test set and then prints the above-mentioned metrics we are interested about.

In [3]:
```python
from sklearn.linear_model import LogisticRegression

from lab_6_util import timeit, evaluate_model

# Time the training of logistic regression classifier
with timeit("Training logistic regression classifier"):
    logistic_model = LogisticRegression(max_iter=200).fit(feat_train, y_train)

# Evaluate trained model and print metrics
evaluate_model(logistic_model, "logistic regression classifier", feat_test, y_test)
```

```
Training logistic regression classifier started...
Training logistic regression classifier completed. Elapsed time: 0.64s

[ Evaluation result for logistic regression classifier ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.89      0.59      0.71       199

    accuracy                           1.00    113923
   macro avg       0.95      0.80      0.86    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113710     14]
 [    81    118]]
```

Voting

The simplest kind of ensemble is a **voting ensemble**. Like a group of people making decisions through a majority vote, a voting ensemble contains multiple classification models, usually implemented from different algorithms. During training, each model learns independently from others. To make a prediction using the ensemble, each model "votes" by providing its own prediction computed from the sample features. The final predicted label of the ensemble is then the class with most "votes" from different models.

For classification models that output a probability distribution over all classes, there is an alternative voting mechanism called **soft voting**. In soft voting, we average the probability of a particular class for all classifiers, and refer to it as the ensemble probability of a class. The ensemble prediction is then the class with the highest ensemble probability. Soft voting largely avoids the **tie-breaking problem** of hard voting, in which two or more majority classes exist with the same number of votes.

In [4]:
```python
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import VotingClassifier

## [ TODO ]
# 1) Create and evaluate a voting ensemble (`VotingClassifier`) with the following su
#    - A logistic regression classifier
#      (Hint: increase maximum iteration to 200 to avoid non-convergence warning)
#    - A Gaussian naive Bayes (`GaussianNB`) classifier
#    - A decision tree classifier

# 2) Evaluate the performance of each sub-classifier on the test set
#    (Hint: obtain the sub-classifiers through `voting_ensemble.named_estimators_`)
# 3) Change the voting mechanism to soft voting. Does the performance of the ensemble
voting_ensemble = VotingClassifier([('lr', LogisticRegression(max_iter=200)),
                                    ('gnb', GaussianNB()),
                                     ('dt', DecisionTreeClassifier())],
                                     voting='hard')
voting_ensemble.fit(feat_train, y_train)
lr = voting_ensemble.named_estimators_.lr.predict(feat_train)
gnb = voting_ensemble.named_estimators_.gnb.predict(feat_train)
dt = voting_ensemble.named_estimators_.dt.predict(feat_train)
# Evaluate the voting ensemble classifier
evaluate_model(voting_ensemble, "voting ensemble", feat_test, y_test)

voting_ensemble2 = VotingClassifier([('lr', LogisticRegression(max_iter=200)),
                                    ('gnb', GaussianNB()),
                                     ('dt', DecisionTreeClassifier())],
                                     voting='soft')
voting_ensemble2.fit(feat_train, y_train)
evaluate_model(voting_ensemble2, "voting ensemble soft", feat_test, y_test)
```

```
[ Evaluation result for voting ensemble ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.80      0.77      0.79       199

    accuracy                           1.00    113923
   macro avg       0.90      0.89      0.89    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113686     38]
 [    45    154]]

[ Evaluation result for voting ensemble soft ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.79      0.76      0.78       199

    accuracy                           1.00    113923
   macro avg       0.90      0.88      0.89    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113685     39]
 [    48    151]]
```

Bagging / Random Forest

Bagging ensemble is a kind of ensemble built upon voting. Compared to regular voting ensembles, a bagging ensemble only contains several classifiers of the **same type** (implementing the same algorithm and using the same hyper-parameter settings), each of which is trained on **a random subset of samples (and / or features)**. Bagging reduces over-fitting of the original classification model by introducing randomization into its construction and then making an ensemble out of it. It works best with strong and complex machine learning models such as neural networks and deep decision trees.

In the following code cell, we will train a few **logistic regression bagging ensemble** with different settings. We will alter the number of (sub-)classifiers, proportion of samples and features and study their influence on the performance of the bagging ensemble:

In [6]:
```python
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import BaggingClassifier

## [ TODO ]
# 1) Create and evaluate a logistic regression bagging ensemble (`BaggingClassifier`)
#    with 10 sub-classifiers, each using 20% of samples.
# 2) Based on the ensemble of question 1, increase the number of sub-classifiers to 3
#    and evaluate again.
# 3) Based on the ensemble of question 1, increase the proportion of training samples
#    each classifier to 60% and evaluate again.
# 4) Based on the ensemble of question 1, enable boostrapping of features, set the
#    proportion of features to 50% and evaluate again.
bagging_models = {
    "10 class 20%": BaggingClassifier(n_estimators = 10, max_samples = 0.2),
    "30 class 20%": BaggingClassifier(n_estimators = 30, max_samples = 0.2),
    "30 class 60%": BaggingClassifier(n_estimators = 30, max_samples = 0.6),
    "30 class 50% boostrap": BaggingClassifier(n_estimators = 30, max_samples = 0.5,
}

for setting, model in bagging_models.items():
    # Train each bagging classifier
    with timeit(f"Training bagging ensemble ({setting})"):
        model.fit(feat_train, y_train)
    # Evaluate each bagging classifier
    evaluate_model(model, f"bagging ensemble ({setting})", feat_test, y_test)
```

```
Training bagging ensemble (10 class 20%) started...
Training bagging ensemble (10 class 20%) completed. Elapsed time: 19.46s

[ Evaluation result for bagging ensemble (10 class 20%) ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.93      0.75      0.83       199

    accuracy                           1.00    113923
   macro avg       0.96      0.87      0.91    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113712     12]
 [    50    149]]

Training bagging ensemble (30 class 20%) started...
Training bagging ensemble (30 class 20%) completed. Elapsed time: 57.54s

[ Evaluation result for bagging ensemble (30 class 20%) ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.93      0.74      0.82       199

    accuracy                           1.00    113923
   macro avg       0.96      0.87      0.91    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113712     12]
 [    51    148]]

Training bagging ensemble (30 class 60%) started...
Training bagging ensemble (30 class 60%) completed. Elapsed time: 201.33s

[ Evaluation result for bagging ensemble (30 class 60%) ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.93      0.79      0.86       199

    accuracy                           1.00    113923
   macro avg       0.97      0.89      0.93    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113713     11]
 [    42    157]]

Training bagging ensemble (30 class 50% boostrap) started...
Training bagging ensemble (30 class 50% boostrap) completed. Elapsed time: 171.79s

[ Evaluation result for bagging ensemble (30 class 50% boostrap) ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.91      0.79      0.85       199

    accuracy                           1.00    113923
   macro avg       0.96      0.89      0.92    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113709     15]
 [    42    157]]
```

In practice, bagging ensemble is often built upon decision tree classifiers. When each decision tree within the ensemble learns from **both a subset of samples and a subset of features**, the resulting bagging ensemble is called a **random forest**. Below code trains and compares the performance of a single decision tree and two random forests with 40 and 100 decision tree classifiers:

In [8]:
```python
import os
from sklearn.ensemble import RandomForestClassifier

# Number of CPUs for ensemble learning methods
N_ENSEMBLE_CPUS = max(os.cpu_count()//2, 1)

# A regular decision tree classifier
with timeit("Training DT classifier"):
    dt_model = DecisionTreeClassifier()
    dt_model.fit(feat_train, y_train)

## [ TODO ]
# 1) Train a random forest classifier with 40 decision trees
# 2) Train a random forest classifier with 100 decision trees
#    (Hint: set `n_jobs` to `N_ENSEMBLE_CPUS` to train the random forest in parallel
rf_40_model = RandomForestClassifier(n_estimators=40, n_jobs=N_ENSEMBLE_CPUS).fit(fea
rf_100_model = RandomForestClassifier(n_estimators=100, n_jobs=N_ENSEMBLE_CPUS).fit(f

# Evaluate previous models
evaluate_model(dt_model, "DT classifier", feat_test, y_test)
evaluate_model(rf_40_model, "Random forest classifier (40 DTs)", feat_test, y_test)
evaluate_model(rf_100_model, "Random forest classifier (100 DTs)", feat_test, y_test)
```

```
Training DT classifier started...
Training DT classifier completed. Elapsed time: 15.87s

[ Evaluation result for DT classifier ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.74      0.74      0.74       199

    accuracy                           1.00    113923
   macro avg       0.87      0.87      0.87    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113673     51]
 [    51    148]]

[ Evaluation result for Random forest classifier (40 DTs) ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.93      0.78      0.85       199

    accuracy                           1.00    113923
   macro avg       0.96      0.89      0.92    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113712     12]
 [    44    155]]

[ Evaluation result for Random forest classifier (100 DTs) ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.94      0.78      0.85       199

    accuracy                           1.00    113923
   macro avg       0.97      0.89      0.93    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113714     10]
 [    43    156]]
```

Boosting

Boosting is an ensemble learning technique that aims to combine **a set of weak learners** (a classifier that is only **slightly better** than a random classifier) into a strong learner. It is able to reduce both the bias and variance of the original classification models. A boosting algorithm usually consists of **iteratively learning weak classifiers** with respect to a distribution and **adding them to a final strong classifier**. Weak classifiers are typically weighted in some way that is related to its performance. After a weak learner is added, sample weights are re-adjusted so that **misclassified samples are stressed** and correctly classified samples are paid less attention to. This causes future weak learners to focus more on samples that previous weak learners fail, thus making the ensemble more robusting against variations in sample features.

In the following code cell, we will try two kinds of boosting ensembles: **AdaBoost and gradient boosting**. Both use one-level (as opposed to full, deep) decision trees as base weak classifiers. AdaBoost adjusts the weights of training samples and weak classifiers based on the accuracy, while gradient boosting adjusts the weights by differentiating through the target loss and computing the corresponing gradient for gradient descent.

In [9]:
```python
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier

# AdaBoost: adjusts weights based on misclassification rates (50 default weak learner
with timeit("Training AdaBoost classifier (50 DTs)"):
    adaboost_model = AdaBoostClassifier()
    adaboost_model.fit(feat_train, y_train)

# Gradient boosting: sequentially corrects errors using gradient descent (40 estimato
with timeit("Training gradient boosting classifier"):
    gb_model = GradientBoostingClassifier(n_estimators=40)
    gb_model.fit(feat_train, y_train)

# Evaluate boosting models
evaluate_model(adaboost_model, "AdaBoost classifier", feat_test, y_test)
evaluate_model(gb_model, "gradient boosting classifier", feat_test, y_test)
```

```
Training AdaBoost classifier (50 DTs) started...
Training AdaBoost classifier (50 DTs) completed. Elapsed time: 60.95s

Training gradient boosting classifier started...
Training gradient boosting classifier completed. Elapsed time: 133.07s

[ Evaluation result for AdaBoost classifier ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.80      0.70      0.75       199

    accuracy                           1.00    113923
   macro avg       0.90      0.85      0.87    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113688     36]
 [    59    140]]

[ Evaluation result for gradient boosting classifier ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.89      0.61      0.72       199

    accuracy                           1.00    113923
   macro avg       0.94      0.80      0.86    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113709     15]
 [    78    121]]
```

Stacking

The last ensemble learning scheme is **stacking**, which is also the most sophisticated one among the four we have introduced today. Like all previous types of ensemble, a stacking ensemble contains a number of classifiers, each possibly using a distinct machine learning algorithm. However, we would perform K-fold **cross validation** during training on each classifier, so each classifier actually has K clones that are trained on different parts of the training samples. The cross validation also provides us with the **validation predictions for all samples**, gathered from the K clones. After training all base classifiers, we collect and concatenate the validation predictions from clones of different classifiers as features, and then **train a meta-classifier** that predicts the sample label for the whole ensemble.

To predict labels for the test set (and any other unseen dataset) samples, we apply the features to all K clones of all classifiers. For the K clones of the same classifier, we **average their outputs** which are usually the probability distribution over all classes. Like the training data, we then **concatenate the averaged outputs** from different classifiers as features, and finally pass them to the meta-classifier to obtain predictions.

In [10]:
```python
from sklearn.svm import LinearSVC
from sklearn.ensemble import StackingClassifier

with timeit("Training stacking ensemble"):
    # Create a stacking ensemble with a logistic regression meta-classifier and three
    stacking_ensemble = StackingClassifier([
        ("Random forest", RandomForestClassifier(n_estimators=40)),
        ("Logistic", LogisticRegression(max_iter=200)),
        ("SVM", LinearSVC(max_iter=1500))
    ], LogisticRegression(), n_jobs=N_ENSEMBLE_CPUS)
    # Train the stacking ensemble
    stacking_ensemble.fit(feat_train, y_train)

# Evaluate the stacking ensemble
evaluate_model(stacking_ensemble, "stacking ensemble", feat_test, y_test)
```

```
Training stacking ensemble started...
Training stacking ensemble completed. Elapsed time: 237.56s

[ Evaluation result for stacking ensemble ]
Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    113724
           1       0.95      0.76      0.85       199

    accuracy                           1.00    113923
   macro avg       0.97      0.88      0.92    113923
weighted avg       1.00      1.00      1.00    113923

Confusion matrix:
[[113716      8]
 [    47    152]]
```

References

1. Ensemble Learning: https://en.wikipedia.org/wiki/Ensemble_learning
2. Ensemble Learning in Machine Learning: https://towardsdatascience.com/ensemble-learning-in-machine-learning-getting-started-4ed85eb38e00
3. Random Forest: https://en.wikipedia.org/wiki/Random_forest
4. Boosting: https://en.wikipedia.org/wiki/Boosting_(machine_learning)
5. AdaBoost: https://en.wikipedia.org/wiki/AdaBoost
6. Gradient Boosting: https://en.wikipedia.org/wiki/Gradient_boosting